Uppsala University

Parallell och Distribuerad programmering

1TD070

Assignment 2: Stencil application

Alva Christensson, Elin Djurberg and Marcus Moritz

April 16, 2026

Contents

1 Problem description **2**

2 Parallelization **2**

3 Performance **3**

4 Discussion **6**

5 References **7**

A Code **8**

A.1 Parallelization 8

A.2 Header File 13

A.3 Makefile 14

A.4 Performance plots 15

1 Problem description

A common operation needed in computer simulations and digital image processing is stencil application. A stencil often needs to be applied a large number of times on a given vector (or matrix). When applying a stencil we compute a new value of an element depending on the values in the neighbouring elements. In this particular assignment we apply the stencil

$$\frac{1}{12h} \cdot v_{i-2} - \frac{8}{12h} \cdot v_{i-1} + 0 \cdot v_i + \frac{8}{12h} \cdot v_{i+1} - \frac{1}{12h} \cdot v_{i+2} \tag{1}$$

which approximates the first derivative. Here our input data will be finite set of N values on the interval $x \in [0, 2\pi)$ which will represent our function $f(x)$. We have applied periodic boundary conditions for this function [1].

2 Parallelization

The parallelization of the given code had its challenges. Firstly we only read the input file into our root process (which we set to process 0). The root process will also have allocated data for the output. To be able to determine the elements per process we need that all processes have access to the number of values we have read and to achieve this we use MPI_Bcast. MPI_Bcast broadcasts one value from one process to all the other processes.

Each process has allocated memory for the elements we are applying the stencil on and also the buffer around. In figure 1 we can see a picture of how the process_memory is structured. We allocate one block of memory and use pointers to point to blocks inside process_memory. The white squares are the elements where we will apply the stencil and the blue squares are the buffers we use so when we apply the stencil to the edges we still have access to the neighbouring elements. The red star shows which element the stencil is applied to. The black stars represent the which elements are used to calculate the new value.

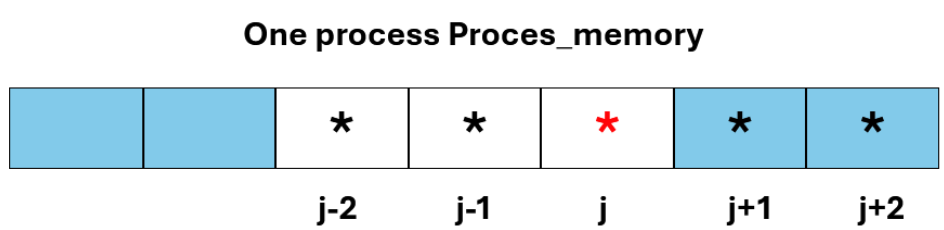

Figure 1: Schematic picture describing how the elements are stored in each process.

To divide the data from the root process we use MPI_Scatter, its here we use elements_per_process so the root process knows how many elements each process should be given [2].

The next part of this problem is updating the buffers before applying the stencil again and as we do not run the stencil over the buffers we need to get the updated values from the neighbouring processes. For this type of problem, persistent communication is well suited as we have the same message size, location, tag, communicator and data type[3]. This is done by first creating two MPI_Requeast arrays. One for the receiving objects and one for the send objects. Next we calculate the rank of the source and the destination but in this case these are the same. We then set up the MPI_Recv_init and MPI_Send_init. We do this one time for the left buffer and one time for the right buffer. In the for-loop where we apply the stencil we start with calling MPI_Startall for both the receiving and sending objects. After that MPI_Waitall is called which waits for all the requests to be done and now we have our updated buffers.

The results when the stencil is applied are stored in a separate array allocated to each process called process_output. After the stencil application is finished, the data in process_output is copied to the corresponding elements in the process memory.

To gather all results in a single process, after all stencil applications have been finished, we use the command MPI_Gather. The pointer to the non-buffer elements are gathered and aggregated onto an output array with the length of all elements. This output array is used to write the results into an output ".txt" file.

3 Performance

To check correctness was checked using the reference output files and these matched exactly. Thus the algorithm is correctly parallelized.

Table 1 describes execution time for different problem sizes and $s = 100$, which means that the stencil is applied 100 times. This was done to get comparable times. For this Table the code was run five times and the best time was chosen.

Table 1: Execution times for $s = 100$ measured in milliseconds.

$p \setminus n$	1000000	2000000	4000000	8000000
1	67.70	159.28	403.58	906.36
2	37.00	86.79	212.37	458.89
4	24.71	52.22	117.60	266.83
8	12.78	40.35	77.78	185.25

To test the speed-up for strong scalability the input file size 8000000 and $s = 100$ was used. In order to calculate the speed-up, Equation (2) was used [4]. The resulting speed-up plotted against the ideal speed-up can be seen in Figure 2.

$$S = \frac{T_s}{T_p} \tag{2}$$

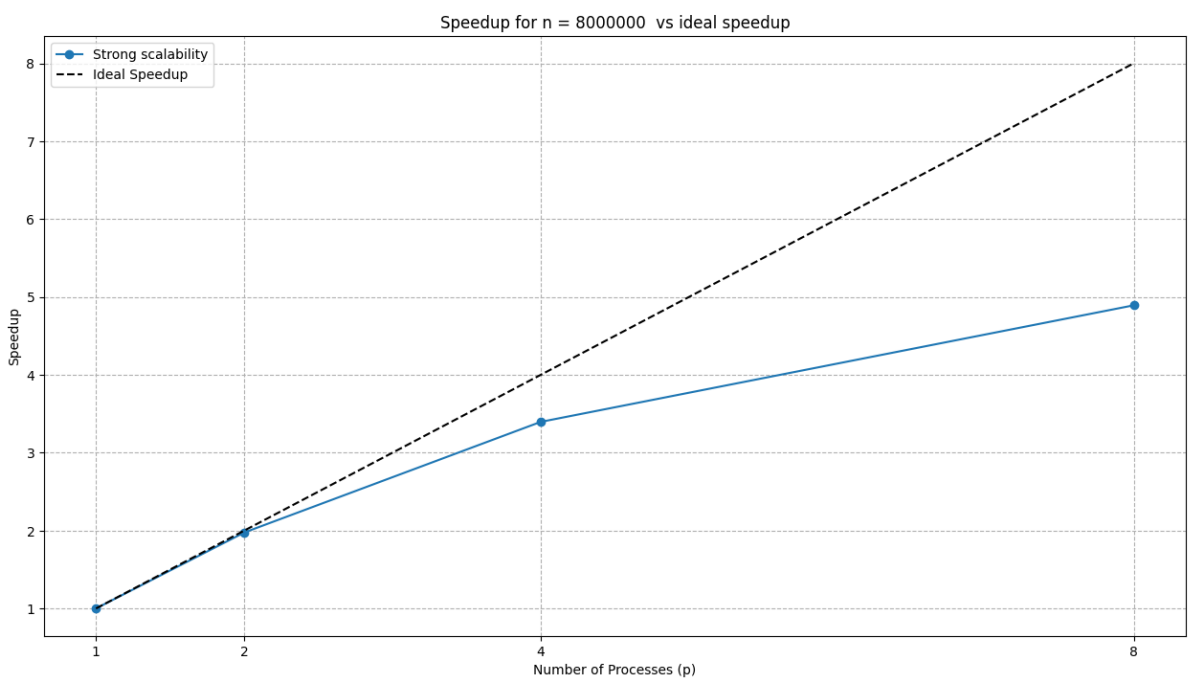

Figure 2: Speed-up for strong scalability.

The weak scalability was plotted by varying the input size, starting with the smallest to the largest while at the same time increasing the number of processors. The derivation $s = 100$ was used here as well. This speed-up compared to the ideal speed-up can be seen in Figure 3. The code for plotting the speed-up for strong and weak scalability can be found in Section A.4.

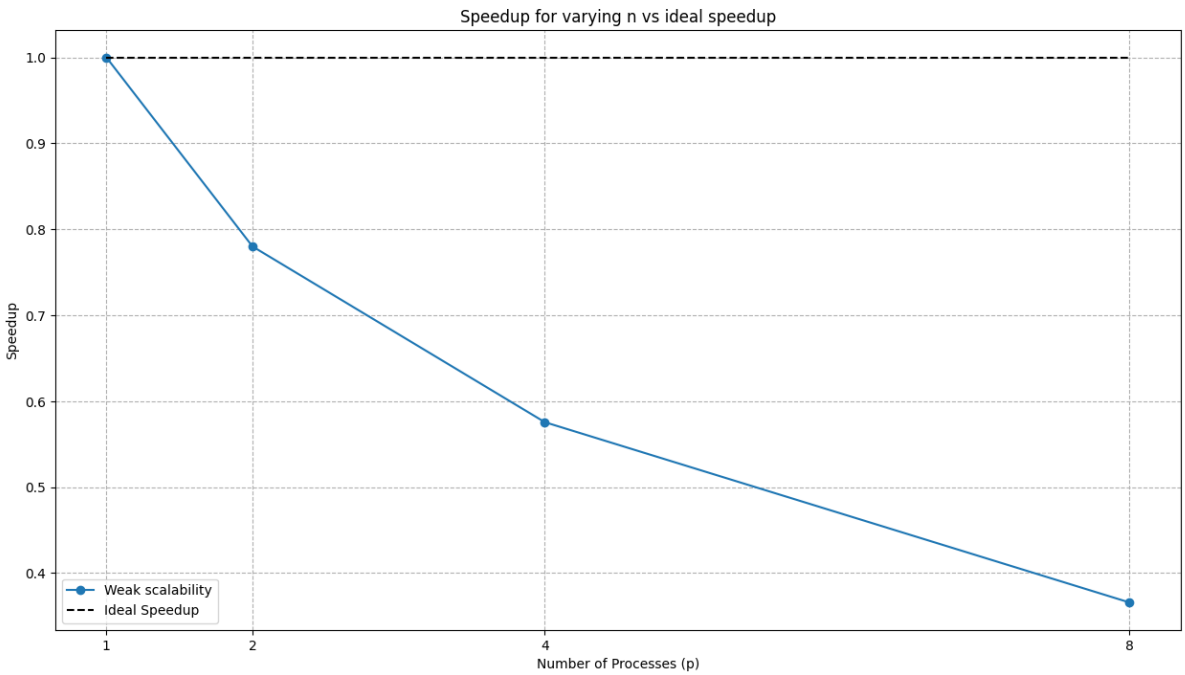

Figure 3: Speed-up for weak scalability.

4 Discussion

From Table 1 a speed-up can be seen when increasing the number of processes for each file size. An issue when performing these runs was that different runs got different values. A reason for this may be caused from using hpc cluster which is limited. Most of the issues can be seen for eight processes and this is most likely a hardware issue.

In Figure 2 the speed-up is very close to the ideal speed-up in for two processors but with weaker speed-up for 4 and 8 processes. It is worthy to note that the scalability is a bit better for the smaller problem sizes, showing that overhead is still significant even for strong scaling.

Figure 3 shows a negative speed-up for more a larger number of processors. This is to be expected for weak scalability since overhead increases with the number of processes, where overhead is more significant for weak scalability.

5 References

[1] R. Iakymchuk, "Assignment no.2," *Studium*, pp. 1–6, 2026.

[2] W. Kendall, *Mpi scatter, gather, and allgather*, Accessed: 2026-04-12, 2026. [Online]. Available: https://mpitutorial.com/tutorials/mpi-scatter-gather-and-allgather/?fbclid=IwY2xjawRNeyNleHRuA2FlbQIxMABicmlkETB0N2hLMExYQlBwbXBzN3Flc3J0YwZ
u6ywp-Ub7fMKUJb67vq4njB50x8Sv1_7zg3VmRbnFc1Utypn_aem_2z_3qq8PHXKIVuG3Qy6IZQ.

[3] G. Collom, J. Burmark, O. Pearce, and A. Bienz, "Persistent and partitioned mpi for stencil communication," in *2024 IEEE High Performance Extreme Computing Conference (HPEC)*, IEEE, Sep. 2024, pp. 1–7. DOI: 10.1109/hpec62836.2024.10938452. [Online]. Available: http://dx.doi.org/10.1109/HPEC62836.2024.10938452.

[4] P. S. Pacheco, *An Introduction to parallel Programming*, English, 1st ed. Amsterdam: Morgan Kaufmann, 2011, ISBN: 9780123742605.

A Code

In the following section the code for the assignment is presented.

A.1 Parallelization

```
#include "stencil.h"
#include <string.h>
#include <stdio.h>


int main(int argc, char **argv) {

   if (4 != argc) {
      printf("Usage: stencil input_file output_file number_of_applications\n");
      return 1;
   }

   char *input_name = argv[1];
   char *output_name = argv[2];
   int num_steps = atoi(argv[3]); //number of times to
   const int root = 0;
   int rank, size;
   MPI_Status status;

   //Setting up MPI
   MPI_Init(&argc, &argv);
   MPI_Comm_size(MPI_COMM_WORLD, &size);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);

   double *input = NULL; //So that each process
   int num_values;

   //Only the the process with rank 0 will read in the full input file
   if(rank == root){
      if (0 > (num_values = read_input(input_name, &input))) {
         return 2;
      }
   }

   //Broadcasting the N number of values to all the process
   MPI_Bcast(&num_values, 1, MPI_INT, root, MPI_COMM_WORLD);
```

```
// Stencil values
double h = 2.0*PI/num_values;
const int STENCIL_WIDTH = 5;
const int EXTENT = STENCIL_WIDTH/2;
const double STENCIL[] = {1.0/(12*h), -8.0/(12*h), 0.0, 8.0/(12*h),
    -1.0/(12*h)};

// Start timer
double start = MPI_Wtime();

double *output = NULL;

//Allocating data for the output file
if(rank == root){
   // Allocate data for result
   if (NULL == (output = malloc(num_values * sizeof(double)))) {
      perror("Couldn't allocate memory for output");
      return 2;
   }
}

//Finding the number of elements per process and declaring pointers for
    storing the results
//of each stencil application for each process
int elements_per_process = num_values/size;
double *process_memory;
double *left_buffer;
double *data;
double *right_buffer;

//Allocating memory for each process
if (NULL == (process_memory = malloc((EXTENT + elements_per_process +
    EXTENT)* sizeof(double)))) {
   perror("Couldn't allocate memory for output");
   return 2;
}

//Setting the pointers to the correct part of the memory
left_buffer = process_memory;
data = process_memory+EXTENT;
right_buffer = process_memory+elements_per_process+EXTENT;

//Scattering the input data from our root process to all the other process
```

```
MPI_Scatter(input, elements_per_process, MPI_DOUBLE, data,
    elements_per_process, MPI_DOUBLE, root, MPI_COMM_WORLD);

//Setting up persistant communication
MPI_Request recv_obj[2];
MPI_Request send_obj[2];

int src; //source and destination are the same

//setting up the left buffer for persistant communication
//if it is the leftmost element the buffer will be filled with the 2
    rightmost elements
if(rank-1 < 0){
   src = size-1;
} else {
   src = rank -1;
}

MPI_Recv_init(left_buffer,EXTENT,MPI_DOUBLE,src, 222, MPI_COMM_WORLD,
    &recv_obj[0]);
MPI_Send_init(left_buffer+EXTENT,EXTENT,MPI_DOUBLE,src, 222, MPI_COMM_WORLD,
    &send_obj[0]);


//setting up the right buffer for persistant communication
//if it is the rightmost element the buffer will be filled with the 2
    leftmost elements
if(rank+1>size-1){
   src = 0;
} else{
   src = rank+1;
}

MPI_Recv_init(right_buffer,EXTENT,MPI_DOUBLE,src, 222, MPI_COMM_WORLD,
    &recv_obj[1]);
MPI_Send_init(right_buffer-EXTENT,EXTENT,MPI_DOUBLE,src, 222, MPI_COMM_WORLD,
    &send_obj[1]);

//Where the process results will be saved
double *process_output = malloc(elements_per_process * sizeof(double));
    //output for each process
```

```
// Repeatedly apply stencil
for (int s=0; s<num_steps; s++) {

   //Starting all the send and recevie requests
   MPI_Startall(2,recv_obj);
   MPI_Startall(2,send_obj);

   //Waits for all the requests to complete
   MPI_Waitall(2, recv_obj, MPI_STATUSES_IGNORE);
   MPI_Waitall(2, send_obj, MPI_STATUSES_IGNORE);

   //Applying the stencil
   for (int i=0; i<elements_per_process; i++) {
      double result = 0;
      for (int j=0; j<STENCIL_WIDTH; j++) {
         int index = i + j;
         result += STENCIL[j] * process_memory[index];
      }
      process_output[i] = result;
   }

   // Swap input and output
   if (s < num_steps-1) {
      memcpy(data, process_output, elements_per_process * sizeof(double));
   }
}

//Gathering all the individual process results in the output file.
MPI_Gather(process_output, elements_per_process, MPI_DOUBLE, output,
    elements_per_process, MPI_DOUBLE, root, MPI_COMM_WORLD);



// Stop timer
double my_execution_time = MPI_Wtime() - start;
double max_execution_time = 0;


MPI_Reduce(&my_execution_time, &max_execution_time, 1, MPI_DOUBLE, MPI_MAX,
    root, MPI_COMM_WORLD);

// Write result
```

```
   if(rank == root){
      printf("%f\n", max_execution_time);
      #ifdef PRODUCE_OUTPUT_FILE
      if (0 != write_output(output_name, output, num_values)) {
         return 2;
      }
      #endif
   }


   //Clean up memory
   free(process_output);
   free(output);
   free(process_memory);
   free(input);

   MPI_Request_free(&recv_obj[0]);
   MPI_Request_free(&recv_obj[1]);
   MPI_Request_free(&send_obj[0]);
   MPI_Request_free(&send_obj[1]);

   MPI_Finalize();

   return 0;
}


int read_input(const char *file_name, double **values) {
   FILE *file;
   if (NULL == (file = fopen(file_name, "r"))) {
      perror("Couldn't open input file");
      return -1;
   }
   int num_values;
   if (EOF == fscanf(file, "%d", &num_values)) {
      perror("Couldn't read element count from input file");
      return -1;
   }
   if (NULL == (*values = malloc(num_values * sizeof(double)))) {
      perror("Couldn't allocate memory for input");
      return -1;
```

```
   }
   for (int i=0; i<num_values; i++) {
      if (EOF == fscanf(file, "%lf", &((*values)[i]))) {
         perror("Couldn't read elements from input file");
         return -1;
      }
   }
   if (0 != fclose(file)){
      perror("Warning: couldn't close input file");
   }
   return num_values;
}


int write_output(char *file_name, const double *output, int num_values) {
   FILE *file;
   if (NULL == (file = fopen(file_name, "w"))) {
      perror("Couldn't open output file");
      return -1;
   }
   for (int i = 0; i < num_values; i++) {
      if (0 > fprintf(file, "%.4f ", output[i])) {
         perror("Couldn't write to output file");
      }
   }
   if (0 > fprintf(file, "\n")) {
      perror("Couldn't write to output file");
   }
   if (0 != fclose(file)) {
      perror("Warning: couldn't close output file");
   }
   return 0;
}
```

A.2 Header File

```
/**
 * This program applies a five point stencil on a vector of function values
 * f(x) for 0<=x<2*PI in order to approximate the derivative of the function.
 * The program takes 3 arguments:
 * - the path to the input file.
 * - the path to the output file. Note that this file will be overwritten!
```

```
 * - an integer specifying how many times the stencil will be applied
 * The formats of the files are described in the documentation of the functions
 * read_input and read_output. The program also prints the number of seconds
 * needed for the stencil application to standard out.
 */

#ifndef _ASSIGNMENT1_STENCIL_H_
#define _ASSIGNMENT1_STENCIL_H_

#define PI 3.14159265358979323846
//#define PRODUCE_OUTPUT_FILE

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

/**
 * Read function data from an input file and store in an array. The input file
 * is supposed to contain an integer representing the number of function values,
 * followed by the function values. All values should be separated by white
 * spaces.
 * @param file_name Name of input file
 * @param values Pointer to array where the values are to be stored
 * @return 0 on success, -1 on error
 */
int read_input(const char *file_name, double **values);

/**
 * Write function data to a file, with 4 decimal places. The values are
 * separated by spaces.
 * @param file_name Name of output file
 * @param output Function values to write
 * @param num_values Number of values to print
 * @return 0 on success, -1 on error
 */
int write_output(char *file_name, const double *output, int num_values);

#endif /* _ASSIGNMENT1_STENCIL_H_ */
```

A.3 Makefile

```
##############################################################################
```

```makefile
# Makefile for assignment 2, Parallel and Distributed Computing.
##############################################################################

CC = mpicc
CFLAGS = -std=c99 -g -O3
LIBS = -lm

BIN = stencil

all: $(BIN)

stencil: stencil.c stencil.h
   $(CC) $(CFLAGS) -o $@ $< $(LIBS)

clean:
   $(RM) $(BIN)
```

A.4 Performance plots

```python
import numpy as np
import matplotlib.pyplot as plt

#processes
p = [1, 2, 4, 8]

#speedup
data_strong = [1, 1.87, 2.0297, 2.0633]

data_weak = [1, 0.9078, 0.9131, 0.2086]

strong_ideal = p #ideal speedup for strong scaling

weak_ideal = [1, 1, 1, 1] #ideal speedup for weak scaling

plt.figure(figsize=(10, 6))

# Plot speedup strong
plt.plot(p, data_strong, marker='o', label='Strong scalability')

# Plot speedup weak
#plt.plot(p, data_weak, marker='o', label='Weak scalability')
```

```
# Plot ideal speedup strong
plt.plot(p, strong_ideal, 'k--', label='Ideal Speedup')

# Plot ideal speedup weak
#plt.plot(p, weak_ideal, 'k--', label='Ideal Speedup')

plt.xlabel('Number of Processes (p)')
plt.ylabel('Speedup')

#Title strong
plt.title('Speedup for n = 8000000 vs ideal speedup')

#Title weak
#plt.title('Speedup for varying n vs ideal speedup')

plt.xticks(p)
plt.grid(True, which="both", ls="--")
plt.legend()
plt.tight_layout()
plt.show()
```